

Aula 15

Matemática em estrutura de dados e na aplicação em algoritmos

Fechamento da disciplina: tudo o que vimos vira código de verdade.

lógica

grafos

árvores

$O(n)$



Nesta aula



Como tudo vira estrutura de dados

Lógica, álgebra booleana, grafos e árvores, todos formalizados de novo em código.



Uma primeira ideia de complexidade

A notação O — como comparar dois algoritmos que resolvem o mesmo problema.



Fechamento da disciplina

O que vem a seguir nas próximas disciplinas do curso.



Amarrando o fio da meada

Cada tema desta disciplina é, na prática, uma estrutura de dados ou ferramenta de análise que você vai reencontrar (e formalizar) em Estrutura de Dados e Algoritmos:

Tema da disciplina	Vira, na prática...
Lógica proposicional	condições (if), invariantes de laço, pré/pós-condições
Álgebra booleana	manipulação de bits, flags, otimizações de baixo nível
Circuitos digitais	por que o hardware "entende" o código que você escreve
Grafos	redes, mapas, dependências, algoritmos de busca (BFS/DFS)
Árvores	listas ordenadas eficientes (BST), hierarquias, parsing



De grafos e árvores a estruturas de dados

Uma estrutura de dados é uma forma organizada de guardar informação para que certas operações sejam rápidas. Muitas das mais usadas são grafos ou árvores por baixo dos panos:



Lista / pilha / fila

Sequências lineares — o caso mais simples de grafo: cada elemento aponta só para o próximo.



Árvore binária de busca

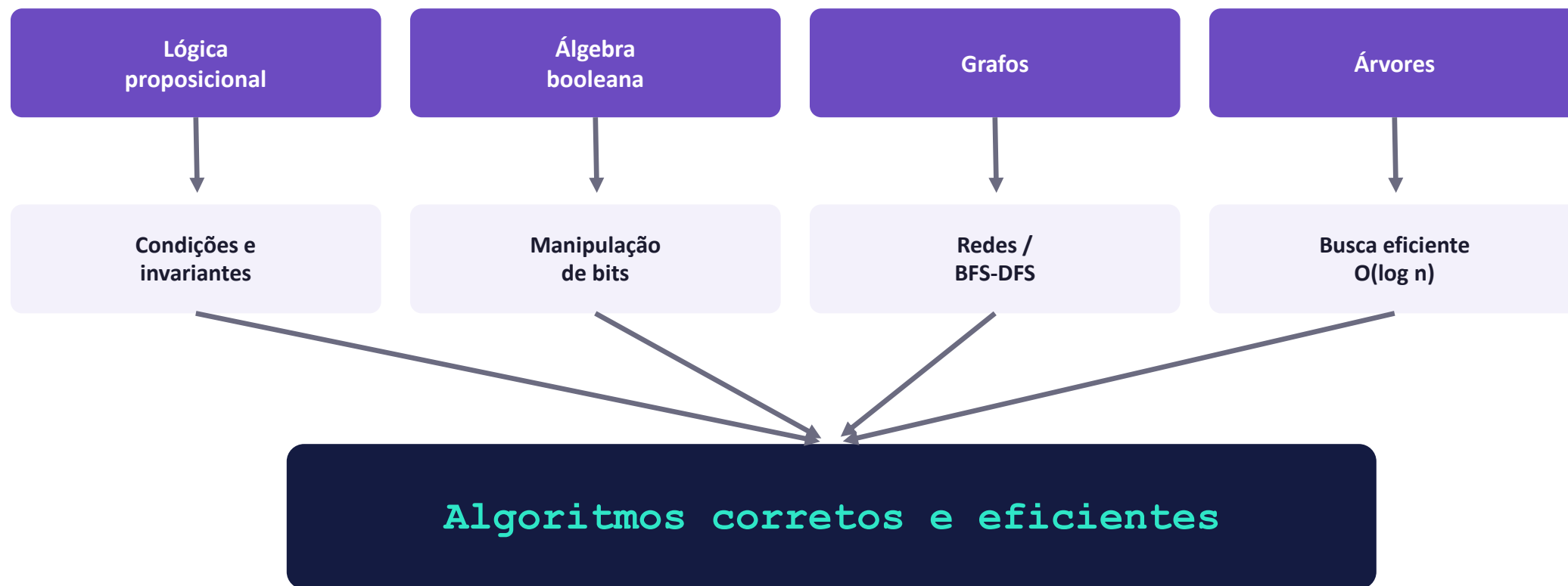
Busca, inserção e remoção em $O(\log n)$ no caso médio, contra $O(n)$ de uma lista (Aula 14).



Grafo de adjacência

Representa redes diretamente (Aula 12) — base de algoritmos de rotas, recomendação e redes sociais.

Da teoria aos algoritmos





Uma primeira ideia de complexidade

Quantas operações um algoritmo faz conforme a entrada cresce? A mesma matemática do semestre, aplicada a código.

Buscar um valor: lista vs. BST

Numa lista de n elementos, buscar no pior caso exige olhar todos os n elementos: $O(n)$.

Numa BST balanceada, cada comparação elimina metade dos candidatos restantes — o mesmo princípio de "descartar metade" de um mapa de Karnaugh bem agrupado. Isso leva a $O(\log n)$ comparações.

Para $n = 1.000.000$: a lista pode exigir 1 milhão de comparações; a árvore balanceada, cerca de 20.

A MESMA IDEIA, EM CÓDIGO

Busca em lista vs. em BST

Cada função conta quantas comparações fez até encontrar o alvo (ou desistir).

Busca.cs



```
static int BuscaLista(List<int> lista, int alvo) {
    int comparacoes = 0;
    foreach (var item in lista) {
        comparacoes++;
        if (item == alvo) return comparacoes;
    }
    return comparacoes;
}

static int BuscaBst(No no, int alvo, int comparacoes = 0) {
    comparacoes++;
    if (no == null || no.Valor == alvo)
        return comparacoes;
    if (alvo < no.Valor)
        return BuscaBst(no.Esquerda, alvo, comparacoes);
    return BuscaBst(no.Direita, alvo, comparacoes);
}
```

busca.py



```
def busca_lista(lista, alvo):
    comparacoes = 0
    for item in lista:
        comparacoes += 1
        if item == alvo:
            return comparacoes
    return comparacoes

def busca_bst(no, alvo, comparacoes=0):
    comparacoes += 1
    if no is None or no.valor == alvo:
        return comparacoes
    if alvo < no.valor:
        return busca_bst(no.esquerda, alvo, comparacoes)
    return busca_bst(no.direita, alvo, comparacoes)
```


COLOCANDO NÚMEROS NA CONTA



1.000.000 vs. ~20 comparações

Para $n = 1.000.000$ elementos: a lista pode exigir 1 milhão de comparações no pior caso. A BST balanceada resolve em cerca de 20 — cada passo elimina metade dos candidatos.

Complexidades comuns

Notação	Nome	Exemplo de operação
$O(1)$	constante	acessar um elemento de uma lista pelo índice
$O(\log n)$	logarítmica	buscar um valor numa BST balanceada
$O(n)$	linear	percorrer uma lista inteira (BFS/DFS também são $O(V+E)$)
$O(n \log n)$	quasilinear	ordenar uma lista com um bom algoritmo de ordenação
$O(n^2)$	quadrática	comparar cada elemento de uma lista com todos os outros

Quanto mais à direita na tabela, mais devagar o algoritmo fica conforme n cresce.

A DIFERENÇA QUE O CRESCIMENTO FAZ



$O(n) \approx 1.000.000$

vs.

$O(n^2) \approx 1.000.000.000.000$

Para $n = 1.000.000$: um algoritmo $O(n)$ faz cerca de 1 milhão de operações; um $O(n^2)$, cerca de 1 trilhão. A diferença parece pequena para $n=10$, mas explode conforme n cresce.



Notação O é sobre crescimento, não um caso só

$O(n)$ e $O(\log n)$ descrevem como o NÚMERO DE OPERAÇÕES CRESCE conforme n cresce — não quantas operações acontecem para um n específico.

É a mesma ideia de generalizar usada para provar que "toda árvore com n vértices tem $n - 1$ arestas": vale para qualquer n , não só para o exemplo desenhado.

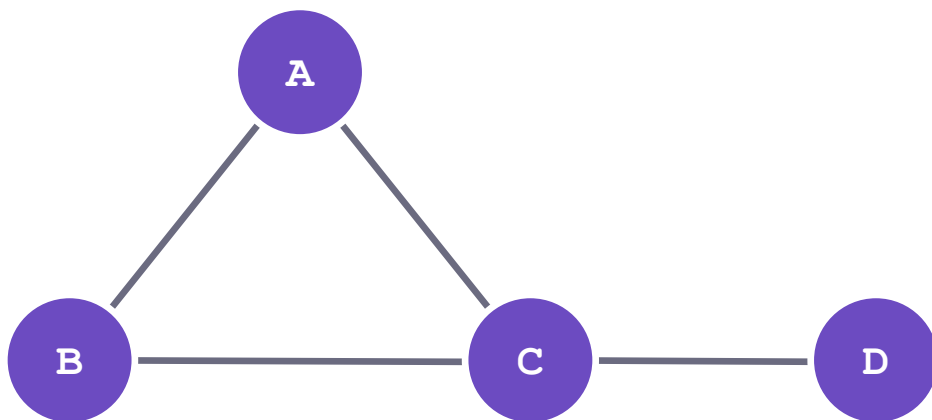


BFS e DFS: dois algoritmos que você já entende

Percorrer um grafo por camadas ou até o fim de um ramo — a mesma travessia da Aula 12, agora com nome.

BFS e DFS

Ambos visitam cada vértice e cada aresta uma vez — por isso rodam em $O(V + E)$, tempo proporcional ao tamanho do grafo.

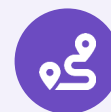


O mesmo grafo A-B-C-D da Aula 12



BFS — Busca em Largura

Usa uma fila. Visita todos os vizinhos de um vértice antes de avançar para o próximo nível — camada por camada.



DFS — Busca em Profundidade

Usa uma pilha (ou recursão, como a travessia em-ordem da Aula 14). Vai até o fim de um ramo antes de voltar.

BFS em código

A partir de um vértice, visita a fila em ordem — quem entra primeiro, sai primeiro.

Bfs.cs



```
static List<string> Bfs(
    Dictionary<string, List<string>> grafo,
    string inicio) {
    var visitados = new HashSet<string> { inicio };
    var fila = new Queue<string>();
    fila.Enqueue(inicio);
    var ordem = new List<string>();
    while (fila.Count > 0) {
        var v = fila.Dequeue();
        ordem.Add(v);
        foreach (var vizinho in grafo[v])
            if (visitados.Add(vizinho))
                fila.Enqueue(vizinho);
    }
    return ordem;
}
```

bfs.py



```
from collections import deque

def bfs(grafo, inicio):
    visitados = {inicio}
    fila = deque([inicio])
    ordem = []
    while fila:
        v = fila.popleft()
        ordem.append(v)
        for vizinho in grafo[v]:
            if vizinho not in visitados:
                visitados.add(vizinho)
                fila.append(vizinho)
    return ordem
```

O que vem a seguir

Esta disciplina deu o vocabulário e o raciocínio de base. As próximas disciplinas do curso vão:



Formalizar notação O com mais rigor

Limites, casos melhor/médio/pior — o que hoje vimos de forma intuitiva.



Implementar como código de produção

Pilhas, filas, listas ligadas, árvores balanceadas e grafos, não só conceito.



Provar que um algoritmo está correto

Invariantes de laço — a mesma lógica formal da Aula 04, aplicada a loops.



REFLEXÃO FINAL · SEM ENTREGA

Encontre a estrutura escondida

Escolha um algoritmo ou estrutura de dados que você já usou em outra disciplina ou projeto (ex.: uma busca, uma fila de tarefas, uma árvore de diretórios) e explique, em poucas frases:

- Que conceito desta disciplina está por trás dele (lógica, álgebra booleana, grafo ou árvore)?
- Por que a versão "ingênua" (sem essa estrutura) seria pior?

Referências

Livros — teoria



CORMEN, T. H. et al.

Algoritmos: Teoria e Prática (CLRS). Elsevier — caps. introdutórios sobre notação assintótica e análise de algoritmos.



GOODRICH, M. T.; TAMASSIA, R.

Estruturas de Dados e Algoritmos em Python. Bookman — visão geral de estruturas e sua análise de custo.



ROSEN, K. H.

Matemática Discreta e suas Aplicações. 7. ed. AMGH/McGraw-Hill — cap. Algoritmos (introdução à complexidade).

Documentação e vídeos



Python — documentação

[collections.deque](#) — fila eficiente, usada no BFS.

[Big-O Cheat Sheet](#) — referência rápida de complexidades comuns.



Vídeos

[CS50 \(Harvard\) — Big-O e estruturas de dados](#)

[mycodeschool — pilhas, filas, árvores, grafos, BFS/DFS](#)

[Introdução a estruturas de dados em Python](#)



FECHAMENTO

A jornada do semestre



Árvore é grafo sem excesso; algoritmo eficiente é matemática sem excesso.

Lógica, álgebra booleana, circuitos, grafos e árvores — o mesmo raciocínio, aplicado de novo e de novo.